

Image Captioning with Attention-based LSTM decoder

Manuel De La Cruz

mxd220071

FREE DAYS USED: 2

Abstract— This paper explores the implementation of an image captioning system that uses deep neural networks that combines pre-trained ResNet-50 Encoder with Bahdanau-style Attention Mechanism and a Long Short Term Memory Decoder that was built from scratch. The encoder works by extracting spatial feature maps of shape $(B, 49, 2048)$ where 49 is the number of patches obtained by the images. The attention mechanism learns to attend to specific regions of the image through learned weights on patches for each decoded word. The custom LSTM cell uses four linear layers, rather than pre-trained modules. The model is trained on the Flickr8k data set, containing 8091 images and 40,455 captions, using teacher forcing and doubly stochastic attention regularization and evaluated using the BLEU-4 metric.

Keywords— image captioning, Attention mechanism, Long Short-Term Memory, encoder-decoder, ResNet-50, Flickr8k, deep learning

I. INTRODUCTION

Automatic Image captioning can be described as the process of generating natural language descriptions of images, sitting at the intersection of computer vision and natural language processing. Applications include accessibility tools for the visually impaired, automated content indexing and image search systems.

The initial works in the early days for image captioning made use of template-based methods with manually made visual features, however thanks to the introduction of deep learning, this has changed a lot. The first trainable model was proposed in [1] which managed to prove the feasibility of training a language model based on visual features while producing high-quality descriptions. The main drawback from this technique however, is in the fact that it results in compressing the whole image into one vector, which makes the caption generation prone to data loss.

Xu et al. [2] addresses this with visual attention in their Attend and Tell framework. Instead of one

singular vector for the entire image, the use of Attention helps the decoder to focus on a selective spatial region of the image when it is deciding which words to generate. This Attention mechanism significantly improved the performance and interpretability of the model.

This project implements an image captioning system based on the idea described by [2], with three key choices being made: a frozen pre-trained ResNet-50[3] encoder, a Bahdanau-style attention mechanism, and a custom LSTM decoder cell that was implemented from scratch. The model will then be trained and evaluated on the Flickr8k dataset [5].

II. THEORETICAL AND CONCEPTUAL STUDY

A. Encoder: ResNet-50 Feature Extraction

The encoder converts the raw input image into feature vectors that focus on specific parts of the image, for the decoder to work over. A pre-trained ResNet-50 [3] trained on ImageNet acts as a backbone to the encoder. The final two layers of the ResNet-50 (Average pooling and classifier), are removed from this process, in order to keep the spatial feature map.

The unique factor of the ResNet is a connection, where the input of each block is summed up to the output. This solves the gradient problem that makes deep networks very difficult to train. It enables gradients to flow through the network without having to transfer through non-linear transformations. As a result of this, ResNet-50 is able to have a good visual feature representation while also being "easy" to train

As mentioned before, the final two layers are removed. These layers get rid of spatial information by breaking the feature map into just one singular

vector. Thanks to this process, the encoder maintains the output of the final convolutional block. An adaptive average pooling layer is then applied to have the spatial resolution at 7×7 even with different input sizes.

The feature map is reshaped to $(B, 49, 2048)$ by treating the grid as independent locations. Each of the vectors are used to represent the visual features of one spatial patch of the image, with 2048 as the dimensionality. This process is very important as instead of one image representation, the encoder produces 49 local representations that the attention mechanism can then treat individually. All the parameters are frozen during the training phase for the sake of computational resources.

B. Attention Mechanism

The attention mechanism is the main part that enhances the system into more than a basic encoder-decoder model. The idea is as follows: when a human describes an image, we do not look at the whole image at once for every word they use, but rather we focus on different parts of the image per word we use. A word like “dog” will be said when focusing on the region with the dog and the word “park” will be when the attention shifts away from the dog and into the environment. The implementation of an attention mechanism helps the model mimic this behavior that we humans do subconsciously.

The implementation followed is with the Bahdanau-style soft attention [2]. In each step of the decoder at time step t , the attention module gets two inputs which are the full set of encoder feature vectors and the current decoder hidden state. These inputs are stored in different vectors and therefore they must be put into a joint space before we are able to compare them.

There are two learned linear transformations that project the features and hidden state into a shared space with attention and dimension 256. The values are summed which makes use of broadcasting the decoder hidden state from the shape $(B, 256)$ to $(B, 1, 256)$, so it adds to each of the encoder patch representations. A $\tanh$ non-linearity is applied, which introduces the ability to model non-linear interactions between the encoder and decoder that a linear model would not be able to replicate.

A final linear transformation brings down the 256-dimensional representation to a single score for each

of the patches, coming out with a vector shape of $(B, 49)$. A softmax function works to normalize the scores into attention weights that must be non-negative and sum up to 1 across all 49 patches. Each weight is representing a fraction of attention devoted to patch i at a decoding step.

The context vector is computed as the weighted sum of the encoder features of all patches. The context vector is a 2048-dimensional summary of certain image regions that are the most relevant to the decoding step. It is concatenated with the word embedding and added to the input to the LSTM cell.

In order for the model to look at every part of the image for the full caption, a doubly stochastic attention regularization term is added to the training loss. This penalizes the stray-away from column-wise sum of attention weights from 1. This makes the model spread out the attention across all the patches for the caption instead of repeating the same subset of patches.

C. Custom LSTM Cell Implementation

Long short-term memory networks [4] were introduced to handle the gradient problem of normal Recurrent Neural Networks. In a standard RNN, gradients have to be propagated across the recurrent weight matrix at each time step. When the matrix has values less than 1, the gradients vanish exponentially based on sequence length, and the learning of dependencies in the distant past becomes practically impossible. To combat this, LSTM introduces a cell state with additive dynamics and mechanisms to control information flow.

This solution implements a custom LSTM cell consists of four nn.Linear layers, corresponding to each individual gate in the cell. The cell has two state vectors at each step. These two are the hidden state, which serves as the cell’s output and the cell state which serves as long-term memory. At each step, the cell receives the concatenated values of the hidden state in the previous step and the current input, forming a vector of $512 + 2304 = 2816$.

The forget gate (f_t) tells us which parts of the cell state could be forgotten and erased. The gate’s components can take values in $(0, 1)$. They multiply the last cell state, and as a result a small value of f_t leads to forgetting that particular cell-state component, whereas a large value indicates that it should be retained. Doing this, we can make the

LSTM model ignore the outdated information and use current/relevant information instead.

The input (i_t) gate is used to calculate how much of the new information will be written into the cell state. The gate (g_t) generates new candidate values that will be then stored into the cell state. The formula $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ combines selective forgetting with selective writing of new information.

The output gate (o_t) controls what the cell state gives up, in order to form the hidden state. The use of tanh on the cell state makes sure that the cell state is in the range $(-1,1)$ before it can be controlled by the output gate. We use the sigmoid activation function for all three gates since its range $(0,1)$ represents the nature of binary control signals.

The additive structure of the cell state (c_t) is what makes the LSTM gradients behave the way they do. There is no multiplication done through the weight matrix, meaning the gradient does not get diluted at each step, allowing the network to learn relations between events separated by dozens of time steps.

D. Decoder Architecture

The decoder produces captions by generating words sequentially in an auto-regressive manner. The initial hidden state of the decoder is determined using the feature representations obtained from the encoder instead of being initialized using zero vectors. The average value of all 49 encoder feature vectors is taken to generate an image summary vector of dimension 2048, which is passed through two distinct linear transformations along with ReLU non-linearity to determine $h_0 \in \mathbb{R}^{512}$ and $c_0 \in \mathbb{R}^{512}$.

At each decoding step t , the decoder executes the following:

The index of the current word I_t is mapped using a learned embedding table of size 256 to create an embedding of word $e_t \in \mathbb{R}^{256}$.

The dropout layer of probability 0.5 is used for regularization purposes on the embedding.

An attention mechanism calculates a context vector based on the encoder outputs and the current hidden state.

The concatenation of the word embedding and context vector is used as the LSTM cell input.

The custom LSTM cell computes a new hidden and cell states.

A dropout layer is added to the output hidden state

The hidden state is mapped using a fully connected layer to a logit vector of size 3,005, representing the vocabulary size.

E. Training Procedure

The model learns with “teacher forcing”, where the correct word at time t is fed as an input at time t while training, rather than using what is the predicted output as input. This way, the model will not experience accumulation of errors, due to the initial predictions of the model being inaccurate. Although the learning process will be made faster, the problem of training/inference mismatch will appear as the model will use its prediction for further learning after training.

The loss function is defined as the cross-entropy loss between the output logits of the target word and the actual word, summed over all time steps in the caption. The padding token is ignored in computing the loss by using `ignore_index = 0`, which works to make sure the padding does not impact the loss. Doubly stochastic attention term is added to the cross-entropy loss.

Adam optimizer[8] is used with a learning rate $= 4 \times 10^{-4}$. The variable learning rates used by Adam let the optimizer work well in this model, where the changing of weights have very different ranges depending on their location in the embedding layer, the attention network, the LSTM, or the output network. Gradient clipping with norm maximum value 5.0 is also applied to prevent exploding gradients from occurring during back propagation.

III. DATASET

The Flickr8k dataset [5] is popular dataset used when working on image captioning and the one I chose for this project. The dataset contains 8,091 images that come from the Flickr platform. The images in the dataset have been chosen to depict a range of different scenes and activities.

Every image in the dataset has five captions associated with it. These captions are from humans and a result of crowdsourcing. The multiple captions are important because it shows the different number of ways the same image can be described by different

people, which allows us to take into account how an image can be perceived. BLEU-4 score is calculated for each image when compared to all the five captions instead of just one.

The text preprocessing phase is composed of lowercasing all the captions and tokenizing whitespace. The generation of the vocabulary is then achieved through a word frequency count in the complete training dataset, where only words that appear five or more times are included. This threshold makes sure to remove infrequent words that the model would not be able to learn due to the lack of training available for them.

Four special tokens are added to this vocabulary, occupying the first four indices. <PAD> takes up index 0 and is used to pad short captions to the length of the longest captions in each set. <START> takes up index 1 and it's used to signal the decoder to begin the generation. <END> takes up index 2 and is used to signal the end of when the generation should stop. Finally, <UNK> takes up index 3 and it works to replace any word that was not found in the vocabulary during encoding.

The images are resized to have the dimensions 224x224 pixels, in order to correspond to the expected input dimensions that are in ResNet-50. The normalization process uses the mean and standard deviation of the imageNet dataset, making sure that the pre-trained encoder receives inputs distributed in the same way as they were during training. Variable-length captions in each batch are padded to the max caption length in that batch using PyTorch's utilities with padding value 0.

IV. RESULTS AND ANALYSIS

The model was trained, and tests were conducted to evaluate the effect of the learning rate and dropout rate on the sensitivity of the model's performance. The experiments used the entire Flickr8k dataset, with a batch size of 64, ten epochs of training, and doubly stochastic attention regularization with $\lambda = 1.0$. The first two experiments ran with the Dropout = 0.5, while the third was changed to 0.3.

Below is the experiment log.

TABLE I. Experimental Results

Exp #	Parameters	Results
-------	------------	---------

1	LR=4e-4, Epochs = 10, Dropout = 0.5, Batch = 64.	Train Loss: 2.76, BLEU-4: 0.0844
2	LR=1e-4, Epochs = 10, Dropout = 0.5, Batch = 64.	Train Loss: 3.150, BLEU-4: 0.089
3	LR=4e-4, Epochs = 10, Dropout = 0.3, Batch = 64.	Train Loss: 2.54, BLEU-4: 0.1069

The table showed training loss decrease across the 10 epochs, verifying that the gradients propagate appropriately throughout the whole neural network model starting from cross-entropy loss, through the output layer, then the custom LSTM cell gates, followed by the attention module, and back to the embedding layer. The decrease nature validates that the custom LSTM cell implementation is working correctly and that the attention regularization does not destabilize training.

In the end, my main parameters which are the ones from experiment 3 gave me the following captions for these three images:

a woman in a pink dress is standing on a bench .



two dogs are playing in the sand .



a dog jumps to catch a frisbee .



As you can observe, the captions generated provided decent descriptions of the images selected, especially in the second and third images.

V. CONCLUSION AND FUTURE WORK

This report discussed a design of an image captioning model that involves a pre-trained ResNet-50 encoder, Bahdanau attention architecture, and a LSTM decoder implemented from scratch. The model was trained on the Flickr8k dataset through teacher forcing, doubly stochastic attention loss, and gradient clipping. This project should meet the requirements of coding the core algorithm from scratch, without the use of any sequence modeling libraries.

The attention mechanism was a valuable enhancement to the baseline encoder-decoder architecture. While the encoder generates only one global vector in the traditional design, the retention of 49 feature vectors from the encoder helps the decoder to selectively pay attention to parts of the image that are relevant at each step. The double stochastic attention regularization helps to address the issue of the model paying attention to a limited number of patches at any time, which is possible when there is no constraint on covering the entire image during the generation process.

The custom LSTM cell proves that the equations for four-gate LSTM equations can be reproduced correctly using independent linear transformations. Using separate linear layers are able achieve similar outputs as the nn.LSTMCell implementation from PyTorch. The division into four gates allows direct correspondence between the code and the equation [4].

As far as future improvements goes, there are a number that I have been able to identify and focus on. The first, would be replacing greedy decoding with beam search as it would be likely to enhance the quality of captions, as beam search takes into account a range of possible sentences and chooses the most probable among them. The second improvement I would make is further fine-tuning the ResNet-50 encoder in the later stages of training, after the decoder has been stabilized, would potentially make the features better suited when generating captions. The third thing I have considered for future work is training the model on a larger dataset such as MS COCO[7]. This dataset is much larger with over 330,000 images at the same 5 captions per image ration. Training on a larger dataset would allow the model to be exposed to more scenes, objects, and descriptions, which should improve the caption generation.

REFERENCES

- [1] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and Tell: A Neural Image Caption Generator," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Boston, MA, 2015, pp. 3156–3164.
- [2] K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhutdinov, R. Zemel, and Y. Bengio, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention," in Proc. Int. Conf. Machine Learning (ICML), Lille, France, 2015, pp. 2048–2057.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, 2016, pp. 770–778.
- [4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [5] M. Hodosh, P. Young, and J. Hockenmaier, "Framing Image Description as a Ranking Task: Data, Models and Evaluation Metrics," Journal of Artificial Intelligence Research, vol. 47, pp. 853–899, 2013.
- [6] K. Papineni, S. Roukos, T. Ward, and W. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL), Philadelphia, PA, 2002, pp. 311–318.
- [7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in Proc. European Conf. Computer Vision (ECCV), Zurich, Switzerland, 2014, pp. 740–755.

- [8] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. Int. Conf. Learning Representations (ICLR), San Diego, CA, 2015.